

CSSE 230 – Data Structures and Algorithms

Exam 3 Spring 2021 Name: _____ CM: _____ Section: 01 04

You may use a calculator, a writing implement, and a 1-sided 8.5" x 11" note page. You must turn in this written part before you use your computer, textbook, or any other resources.

You may not communicate with any person other than your instructor about this exam until allowed by your instructor. You may not use electronic devices.

Part 1 scores (for instructor use):

Problem	Possible	Score
1	10	
2	15	
3	12	
4	15	
5	8	
6	5	
Total	65	

Computer part = 55 points

Total	120	
--------------	------------	--

Part 2 (computer) rules

You may use any printed or written materials that you brought with you, code that you wrote yourself or with a partner before the exam, anything on the CSSE230 Moodle and web sites, and the Java API documentation (on your laptop or at oracle.com).

You may not otherwise search the internet or communicate with any person other than your instructor or a student assistant. You may not communicate with anyone else about this exam until allowed by your instructor. You may not use other electronic devices.

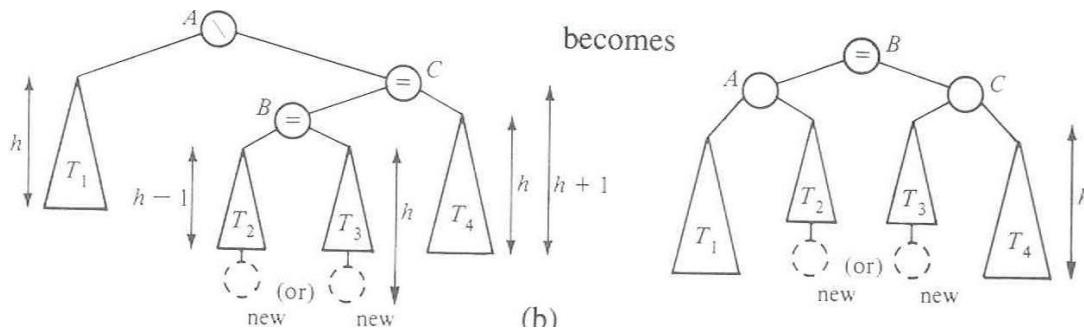
You will sign a statement to certify that you neither received help from others nor gave it on either part of the exam.

You must actually get these problems working on your computer. All or most of the credit for these problems will be for code that actually works.

1. (10 points) T/F/IDK. Below you will find several statements. A statement is true (T) if it is always true. It is false (F) if there is at least one counterexample (sometimes false). You may also choose IDK to indicate that you do not know the answer. **Point values:** Correct answer: 2, incorrect answer: -1, IDK: 1, blank: 0. **Circle at most one answer for each part.**

- a. T F IDK After insertion in an Editor Tree, you need to recalculate the height at some nodes to determine the new balance code.
- b. T F IDK Insertion in a height-balanced tree never needs more than one single or one double rotation to fix an imbalance.
- c. T F IDK The array in a binary min heap is always in ascending sorted order.
- d. T F IDK It is possible in a hash table for $\lambda = 10$.
- e. T F IDK Fixing an imbalance in a height balanced tree, after inserting a value and an imbalance is found, is $O(1)$.

2. (15 points) Recall that in a height-balanced tree, that *double* rotations are called that since they can be implemented using two single rotations. (a) (10 points) Write the following **doubleRotateLeft()** method meant to be called after a node inserted under T2 or T3 causes an imbalance at A (see diagram). For full credit, your solution must:
- call the given singleRotateLeft() and singleRotateRight() methods correctly, that is, by calling them on the correct nodes in the correct order and using their return values correctly.
 - correctly set any node's balance codes that weren't fixed within the single rotate methods.
 - return the new root of the double-rotated subtree.



The first two are implemented for you per the comments and are meant to be called:

```
class Node {
    // Does a single left rotation, so reassigns Node references, updates 2 ranks
    // appropriately and 2 balance codes to SAME. In Node, so "this" is the unbalanced node.
    // Returns its right child since it's the new root of the rotated subtree.
    public Node singleRotateLeft()
    public Node singleRotateRight() // Same, but a single right rotation

    // Write the body of this one, as specified above:
    public Node doubleRotateLeft() {
```

(b) (5 pts) Suppose R_A , R_B , R_C are the "old" ranks of A, B, and C. Using these, give formulas for the "new" ranks after the double left rotation.

New rank of A = _____ New rank of B = _____ New rank of C = _____

3. (12 points) Consider the usual hashCode function for integers, $h(k) = k$. The hash code is taken modulo the table size, M , to find the initial probe index in the table.

(a) (4 points) Linear probing. Below is an empty hash table with $M = 13$. Insert each integer from the list
5, 9, 18, 12, 25, 38

in that order, using linear probing to resolve collisions. Show where each number lands in the table, and beneath it, how many probes it took to reach there. (Count the hashed-to index as the first probe.)

Be careful with the arithmetic. Don't grow the array.

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Number inserted													
# of probes required													

(b) (2 points) Consider instead using quadratic probing. If for an integer J , $h(J) \% M = x$, but positions x and $(x+1)$ are both already occupied, which position will be probed next to add J ? _____

(c) (2 points) We learned a theorem guaranteeing insertion while inserting elements using quadratic probing. Does that theorem guarantee insertion of 10 unique elements into the above array?

Circle one: [yes | no] and explain why/why not:

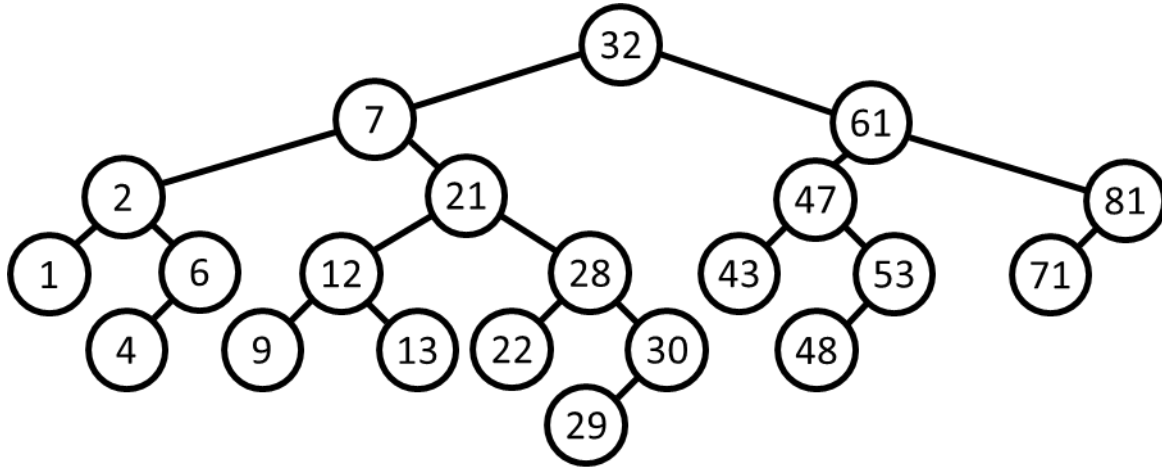
(d) (2 points) Consider instead using separate chaining on the above table of size 13. After inserting 10 unique elements....

What is the load factor? _____

What is the average length of the linked lists? _____

(e) (2 points) If we have a (poorly-chosen) hash function, $h(x) = 5$ for all x , then what is the worst-case big-Theta runtime of `contains()`? _____

4. (15 points) Consider the AVL tree below.
 (a)-(d) Fill in the table below. For each part, **begin with the original tree**.



			single, double, or none?	left or right?	root of rotated subtree before rotation (i.e., "imbalanced node")	root of rotated subtree after rotation
Example	insert 70	1st rotation	single	right	81	71
		2nd rotation	none			
part (a)	insert 5	1st rotation				
		2nd rotation				
part (b)	delete 6	1st rotation				
		2nd rotation				
part (c)	delete 43	1st rotation				
		2nd rotation				

Reminder: did you start with the original tree for each part?

5. (8 points) Draw a tree for the following pairs of tree-based definitions that satisfies both. Label nodes with integers for the data contained in each node. You must use **at least 10 nodes in total** – so if you use 3 for (a), you need to use at least 7 for (b).

(a) Binary Search Tree and Binary Min Heap

(b) Height-Balanced Tree and Binary Min Heap

6. (5 points) Consider a binary tree with height 4. What are the:

a. Maximum number of nodes it could have? _____

b. Minimum number of nodes it could have? _____

c. Minimum number of nodes it could have and still be height-balanced? _____

You may use any printed or written materials that you brought with you, code that you wrote yourself or with a partner before the exam, anything on the CSSE230 Moodle and web sites, and the Java API documentation (on your laptop or at <http://download.oracle.com/javase/7/docs/api/>).

You may not otherwise search the internet or communicate with any person other than your instructor or a student assistant. You may not communicate with anyone else about this exam until allowed by your instructor.

You must actually get these problems working on your computer. All of the unit test points credit will be for code that actually works. Instructions:

1. Read the problems on the next page
2. Solve and commit your work.

Turn-in checklist:

1. _____ I did not and will not give or receive help on this exam. Signature: _____
2. _____ I saved my work and committed my solution to my SVN repository.
3. _____ I verified in Eclipse that my solution was committed.
4. _____ I am about to hand this test paper, with my name and section number on it, to my instructor.

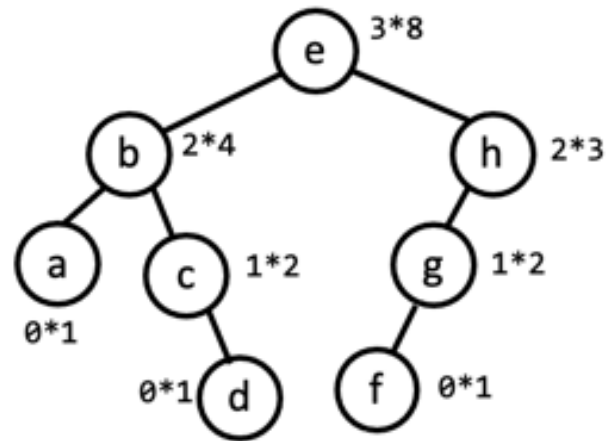
Instructor use only:

Problem	Possible	Score	Comments
1. weightedHeight	20		
1. weightedHeight efficiency	15		
2. numCollidedEntries unit tests	20		
Total	55		

7. (35 points = 20 unit test + 15 efficiency)

Implement `int weightedHeight()`. The weighted height of a single **node** is simply the product of its height and its size (both as defined in this course). The weighted height of any **tree** is the sum of the weighted heights of all the nodes in the tree.

For the example tree `t` to the right, the height*size is listed for each node. The sum of all of these products is 42, so `t.weightedHeight()` would return 42.



For the efficiency points, the method must run in $O(N)$ time and use only a **single traversal** of the tree, visiting each node once. It may not use any extra data structure whose size depends on the size of the tree (other than the stack that Java uses to keep track of recursive calls, of course). You may not include any other fields in the `BinarySearchTree` or `BinaryNode` classes or modify the code we used to build the tree. However, you may add helper methods with parameters and return values of any type, including any custom class (like a container) that you define.

8. (20 points) **StringHashSet numCollidedEntries**. In the `hash` package, implement the method `int numCollidedEntries()`. We simplified the implementation of the `StringHashSet` to use linear probing and to not grow the array and re-hash entries. You are to return the number of entries that must have collided with at least one other item when inserted. You can determine it by writing code to examine the internal hash array using knowledge of how linear probing works.

Note: do not re-run `add()` or `contains()` to determine this –changing either of those methods or adding fields to the class is not allowed.

You may refer back to your implementation of the `StringHashSet` assignment as needed.